Bootstrapping Vision-Language-Action Models for New Robot Embodiments with Staged Reinforcement Learning

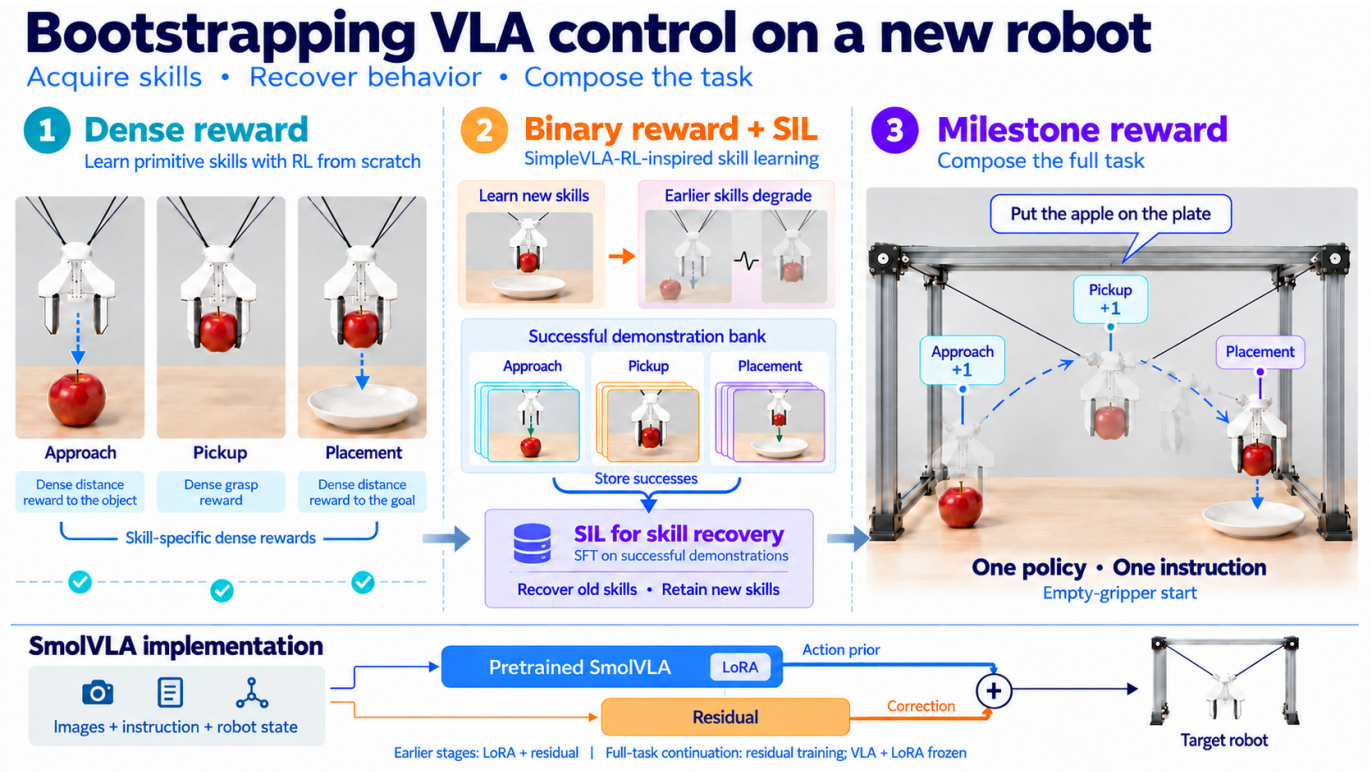


Fig. 1. Proposed pipeline and SmolVLA implementation. Dense feedback acquires skills; binary refinement and balanced self-imitation consolidate them; milestone-based RL learns the complete task under one instruction. Skill-specific resets are used during acquisition. Full-task evaluation starts with an empty gripper and uses no external switching between specialists.

WORKING DRAFT — 29 October 2026

***Abstract*—A pretrained vision-language-action (VLA) model can produce few successful rollouts when transferred to a new action interface. Sparse rewards then hinder learning, while sequential skill training can erase earlier behavior. We propose a pipeline of dense skill acquisition, binary refinement with balanced self-imitation, and milestone-based full-task composition. Intermediate feedback guides exploration, successful experience consolidates skills, and milestone credit connects them under one instruction. Full-task continuation uses action likelihoods consistent with the sampled continuous commands. We implement the pipeline with a SmolVLA residual controller and LoRA, and examine reward design with OpenVLA-OFT on LIBERO-Spatial. The CDPR controller achieves 70.61% strict success in simulation. On LIBERO-Spatial, dense feedback reaches 65.3% success versus 61.7% with terminal-only reward at the same update budget.**

***Index Terms*—Robot learning, reinforcement learning, manipulation, vision-language-action models, robot adaptation.**

I. Introduction

Vision-language-action (VLA) models provide useful visual and language representations for robot control. Transferring these representations to a new robot still requires learning how its actions affect the scene. If the initial policy rarely completes the task, binary success provides little guidance for reinforcement learning.

Consider *put the apple on the plate*. A failed rollout may contain a correct approach and grasp followed by a poor release. Treating the entire attempt as a failure discards this distinction. Training the parts separately helps exploration, but a policy can forget earlier skills while learning later ones. It must then connect the retained skills into a complete behavior.

We address these problems through the pipeline in Fig. 1. Dense rewards first guide skill acquisition. Binary refinement then targets completion, while balanced self-imitation retains successful behavior across instructions. Finally, milestone-based RL trains one policy to execute the full task from an empty-gripper start.

The pipeline separates feedback design and skill consolidation from the robot-specific action mapping. We implement it with SmolVLA [1], LoRA, and a residual action controller. Experiments examine complete-task learning and skill retention in cable-driven parallel robot (CDPR) simulation, and

intermediate feedback with OpenVLA-OFT [2] on LIBERO-Spatial. An ongoing WX200 study extends the evaluation to a 5-DoF serial manipulator.

We contribute a training schedule that combines dense skill acquisition, binary refinement with instruction-balanced self-imitation, and full-task learning under one instruction. We specify milestone credit that retains local credit for achieved approach and pickup stages when later placement fails. Our experiments measure skill recovery after sequential training, report complete-task performance after action-interface adaptation, and compare intermediate and terminal feedback on LIBERO-Spatial.

II. Related Work

Group-relative RL and dynamic sampling. GRPO estimates policy advantages by comparing returns within groups of sampled trajectories [3]. DAPO introduces dynamic sampling that filters groups with uniform binary outcomes and collects additional samples to maintain an informative training batch [4]. SimpleVLA-RL adapts group-relative optimization to interactive robot rollouts, using dynamic sampling and exploration enhancements to improve task execution and discover behaviors beyond the supervised demonstrations [5]. These methods provide a basis for learning task-specific behavior through interaction. We use DAPO-style group filtering and resampling during skill learning.

Alternating RL and supervised learning. iRe-VLA alternates online RL on a lightweight action head with a frozen vision-language backbone and supervised updates using expert data together with successful online trajectories [6]. This separates exploration from supervised consolidation while allowing the large model to benefit from collected experience. Our self-imitation step fits an instruction-balanced mixture of earlier and current successes to retain skills during sequential training.

Residual RL and distillation. Probe, Learn, Distill (PLD) trains lightweight residual specialists over a frozen VLA, collects trajectories through policy-aligned residual interventions, and distills the resulting data into the generalist [7]. We adopt the residual-learning principle to adapt the target action interface. Our training procedure links skill acquisition and successful-experience consolidation before learning to execute the complete instruction.

III. Methods

The goal is to adapt a pretrained VLA to complete language-conditioned manipulation tasks on a target robot. The policy receives camera images, proprioception, and an instruction, predicts an action chunk, and executes a prefix before observing again. We evaluate complete-task success independently of the reward used for training.

During acquisition, each skill has its own instruction, success condition, and starting states. For example, placement can begin with an object already held. During composition and final evaluation, the robot starts with an empty gripper and receives one complete-task instruction. This distinction prevents isolated skill performance from being treated as full-task success.

Algorithm 1 Staged VLA adaptation

Require: Pretrained VLA, skill instructions, fixed evaluations
Initialize the target action controller and success banks
for reward mode: dense acquisition, then binary refinement **do**
  for skill: approach, pickup, placement **do**
    repeat
      Collect same-scene rollout groups for the skill
      Update with $\mathcal{L}_{\mathrm{RL}}$; retain successful trajectories
      Evaluate skill success
    until target, plateau, or stage budget ends training
    Evaluate retention of all learned skills
    if earlier skills have degraded **then**
      Balance earlier and current successful experience
      Fit $\mathcal{L}_{\mathrm{SI}}$; validate recovery and retention
    end if
  end for
end for
repeat
  Collect full-task rollouts from empty-gripper starts
  Assign milestone credit; protect achieved prefix skills
  Update with $\mathcal{L}_{\mathrm{RL}}$; evaluate complete-task success
until target, plateau, or full-task budget ends training
if composition plateaus and budget remains **then**
  Freeze VLA/LoRA; retain the bounded residual
  Reset optimizer; keep milestone rewards unchanged
  repeat
    Collect full-task groups; execute clipped commands
    Store pre-clipping samples and episode offsets
    Score samples conditional on their stored offsets
    Assign milestone credit; protect achieved prefix skills
    Update residual and exploration with $\mathcal{L}_{\mathrm{RL}}$
    Evaluate complete-task success
  until target, renewed plateau, or budget exhaustion
end if
return One policy for the complete instruction

A. Dense Acquisition, Binary Refinement, and Self-Imitation

We first train approach, pickup, and placement skills with dense rewards. Distance reduction and manipulation events guide exploration before skill success becomes common. After dense acquisition, we refine the skills using binary success rewards.

Successful trajectories are retained by instruction. When later training degrades earlier skills, self-imitation fits a balanced mixture of earlier and current successes:

$$\mathcal{L}_{\mathrm{SI}} = \mathbb{E}_{\mathrm{success}} \left[\|\hat{a} - a\|^2 \right]. \tag{1}$$

Here $\hat{a}$ is the predicted action and a is the recorded executed action. The average balances instructions and excludes unused action slots. Policy-dependent features are refreshed before fitting. Balancing matters because imitating only the newest skill can reinforce forgetting.

Algorithm 1 summarizes the schedule. Progress is checked with fixed skill evaluations. After an initial training period, automated stages can end at a success target or a plateau; budget exhaustion also ends a stage. The CDPR sequence used validation-guided manual transitions. After consolidation, all learned skills are evaluated again. Automated runs retain the consolidation update only when recovery improves the degraded skills on average and keeps other skills within tolerance.

B. SmolVLA with an Embodiment-Specific Residual

Figure 2 shows one implementation of the pipeline. Following the residual-learning ingredient of Probe, Learn, Distill (PLD) [7], we retain a pretrained expert and adapt its actions with a lightweight residual. The SmolVLA expert supplies an action prior, mapped to the target control interface. In our five-channel controller, eight action slots give

$$p \in \mathbb{R}^{40} \qquad (8 \text{ slots} \times 5 \text{ channels}). \tag{2}$$

A frozen visual projection supplies 512 features; six proprioceptive values complete the 518-dimensional observation. The residual receives this observation and the 40-dimensional prior, and predicts a bounded correction. The combined action is bounded before execution.

LoRA adapts the action expert during earlier training. In the evaluated full-task continuation, the inherited VLA and LoRA weights are frozen while the residual and exploration parameters are trained. The backbone, action mapping, and feature dimensions can change with the embodiment; the training schedule remains the same.

C. Milestone-Based Full-Task Composition

After skill consolidation, one policy learns the complete instruction from empty-gripper starts. No external controller switches between specialist policies. Approach, held pickup, and final placement serve as milestones for assigning credit within a rollout.

The main rule is simple: a later placement failure should not remove the local credit for a successful approach or pickup. We compare milestone outcomes within groups of rollouts from the same scene. Actions receive credit for the milestone they contribute to, while full-task success provides an additional preference for completing the chain. A negative completion bonus is suppressed on already achieved approach and pickup milestones. Placement receives its completion credit once.

This rule preserves useful partial behavior while training the remaining task. It controls the assigned credit; it does not prevent interference through shared network parameters.

D. Stabilized Updates and Full-Task Continuation

We use a clipped group-relative policy loss with entropy regularization:

$$\mathcal{L}_{\mathrm{RL}} = \mathcal{L}_{\mathrm{clip}} - \eta\mathcal{H}. \tag{3}$$

The clipped loss compares rollouts within each scene group and limits the incentive for large changes in action likelihood. We use separate upper and lower clipping bounds. The entropy term $\mathcal{H}$, weighted by η, maintains exploration during manipulation and composition. Together, these choices help keep early updates conservative while successful behavior is still rare.

During skill learning, DAPO-style dynamic sampling filters uninformative groups and collects additional rollout groups within the configured collection budget [4]. With binary rewards, uniform outcomes provide no group-relative contrast. During dense acquisition, filtering instead checks within-group return variation, so partial progress can provide a learning signal before a group contains a successful skill execution.

Milestone rewards alone need not resolve a full-task plateau. Our CDPR continuation retains the learned bounded residual, freezes the inherited VLA and LoRA, and starts a fresh optimizer while preserving the milestone rewards. The key change is to compute likelihoods from the Gaussian commands sampled before clipping, conditioned on the realized exploration offset. For an episode offset B and its phase gate g_t, we store $b_t = g_t B$ and sample

$$u_t \sim \mathcal{N}(\mu_\theta(x_t, p_t) + b_t, \operatorname{diag}\sigma_\theta^2), \quad a_t = \operatorname{clip}(u_t, -1, 1). \tag{4}$$

The robot executes a_t, while old and new policy likelihoods score the stored u_t conditional on the same b_t. This avoids evaluating a Gaussian density at a clipped boundary [8]. Conditioning also accounts for the offset persisting across decisions, instead of treating it as independent extra noise at every step. The residual and exploration scale remain trainable. Algorithm 1 includes this continuation after a composition plateau; Section IV-A reports complete-task performance. Self-imitation uses the regression loss in Eq. (1).

E. Intermediate Feedback for LIBERO Composition

The LIBERO study asks whether intermediate feedback remains useful when a pretrained policy already performs manipulation skills. We fine-tune an OpenVLA-OFT-based policy with SimpleVLA-RL and LoRA on complete LIBERO-Spatial instructions [2], [5], [9]. This tests the feedback component of the approach when a conventional robot action interface is already available.

We compare terminal-only reward, phase rewards for completed subgoals, and dense feedback that also resolves progress within a phase. A tracker follows reach, grasp, lift, move, and place. Within the active phase, dense credit is given only when progress exceeds its previous maximum; revisiting the same position earns no repeated credit. The progress baseline resets at each phase transition.

Shaping is bounded and added to the terminal reward, while the native task-success verifier remains unchanged. Optional filtering selects rollout groups by their fraction of terminal successes, independently of shaping. We evaluate both the full suite and a separate low-success task (Section IV-C). These comparisons test whether progress feedback helps compose available skills under one instruction, as well as supporting learning when terminal success is rare. The motivation follows from a simple group calculation. For G independent rollouts from one scene, each with success probability p, the probability of observing both success and failure is

$$P_{\mathrm{mixed}} = 1 - p^G - (1-p)^G. \tag{5}$$

With eight rollouts, this probability is only 21.6% at 3% success, but 94.2% at 30% success. Milestones can therefore provide informative comparisons before complete-task success becomes common. Dense feedback can also distinguish all-failure rollouts. A filter that removes such groups discards this potential signal.

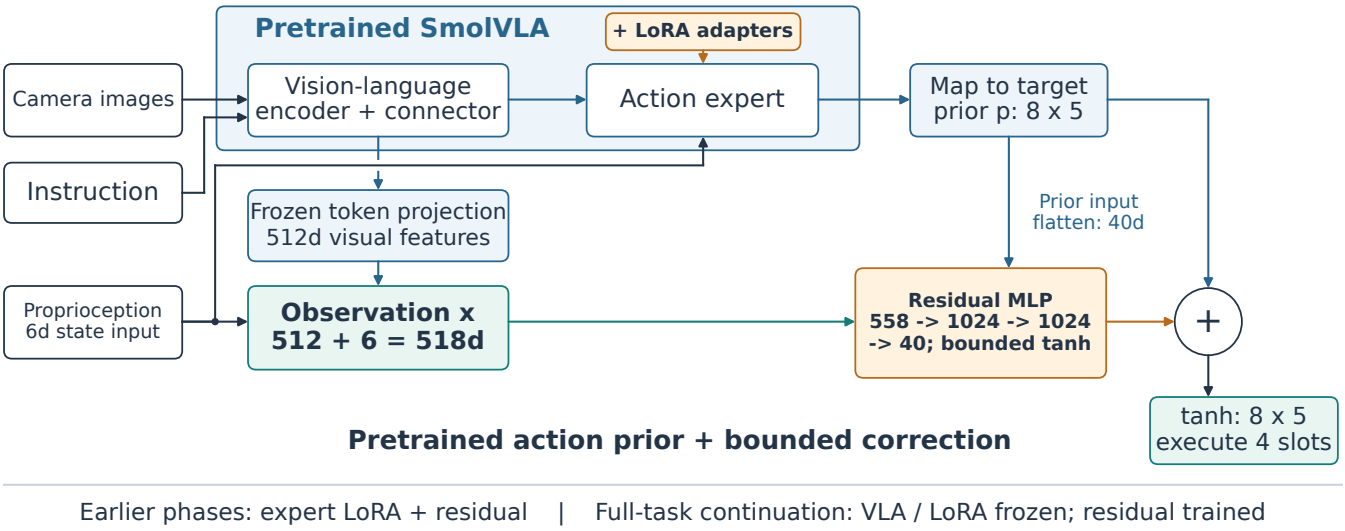


Fig. 2. SmolVLA implementation of the pipeline. The added residual combines 518 observation features with a 40-dimensional action prior. LoRA adapts the pretrained expert during earlier training; the evaluated full-task continuation freezes the VLA and LoRA weights. Feature and action dimensions refer to the five-channel controller.

IV. Experiments

We evaluate three aspects of the approach: full-task composition, retention during sequential learning, and intermediate feedback. SmolVLA experiments test the adaptation pipeline in CDPR simulation. OpenVLA-OFT experiments test reward design on the ten LIBERO-Spatial tasks. The ongoing WX200 study examines another embodiment.

A. SmolVLA: Complete-Task Adaptation

The CDPR simulation uses MuJoCo Warp/MJ-Lab, four object categories (apple, orange, potato, and tomato), and plate or bowl destinations. The action interface controls Cartesian translation, yaw, and the gripper. Strict success requires grasp, held lift, intentional release, and correct placement, without carry slip or wrong-placement events.

The development validation comprises 1,024 full-task episodes: 512 plate and 512 bowl attempts. Each attempt starts with an empty gripper and uses one full-task instruction, without teacher assistance. All attempts, including numerical failures, remain in the denominator. The adapted controller completes 723 of 1,024 development validation attempts (70.61% strict success; Table I). Physical grasp occurs in 92.48% of attempts and held lift in 88.28%. Completion rates are 393/512 (76.76%) for plates and 330/512 (64.45%) for bowls. Carry slips (16.70%) and wrong placements (14.16%) remain the main failure modes.

TABLE I
CDPR complete task development evaluation.

Outcome	Attempts (%)
Physical grasp	92.48
Held lift	88.28
Strict completion (723/1,024)	70.61
Carry slip	16.70
Wrong placement	14.16

B. Skill Recovery with Balanced Self-Imitation

Balanced self-imitation raises reaching success from 8.0% to 31.1% (Fig. 3). Plate placement changes from 50.0% to 52.3%, and bowl placement from 23.05% to 27.65%. The bank combines earlier reaching successes with current placement successes. Each skill is evaluated under its own fixed reset protocol; placement skill tests can begin with a held object.

Recovery is partial, with placement performance retained while reaching improves. These skill-level results do not isolate the contribution of self-imitation to final full-task success.

C. OpenVLA-OFT: Intermediate Feedback on LIBERO

Using the feedback variants in Section III-E, aggregate runs on the ten LIBERO-Spatial tasks share a 61.0% initialization and use 300 validation rollouts per endpoint. The primary dense and terminal-only conditions share the terminal reward weight and 160-update budget. A separate low-success task starts at 2.0%, measured over 1,000 rollouts; both conditions there use the same terminal-outcome group filter. At matched budgets, dense feedback exceeds terminal-only reward by 3.6 percentage points on the suite and 7 points on the low-success task (Table II). This supports intermediate feedback both when success is rare and when a policy already performs many tasks, consistent with its role throughout skill learning and composition. The remaining conditions use different budgets or shaping weights and do not establish a general efficiency advantage.

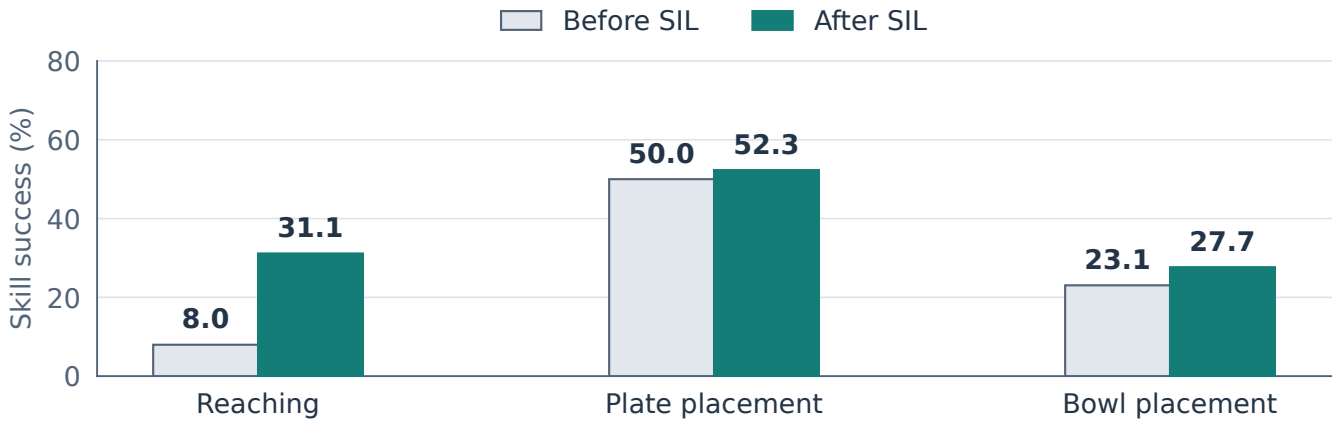


Fig. 3. Skill success before and after balanced self-imitation. Bars show means across three evaluation rounds for reaching and two rounds for each placement skill, per condition. Reaching partially recovers while both placement means increase.

TABLE II
LIBERO REWARD COMPARISON. λ IS THE TERMINAL REWARD WEIGHT; SUCCESS AND Δ ARE PERCENTAGES AND PERCENTAGE-POINT GAINS.

Configuration	Updates	Success	Δ
LIBERO-Spatial: initial 61.0%			
Terminal only, $\lambda = 5$	160	61.7	+0.7
Dense, $\lambda = 5$	160	65.3	+4.3
Filtered dense, $\lambda = 5$	50	64.7	+3.7
Dense, $\lambda = 1$	110	61.0	+0.0
Phase only, $\lambda = 1$	110	62.3	+1.3
Low-success task: initial 2.0% (1,000 rollouts)			
Terminal only	20	11.0	+9.0
Dense	20	18.0	+16.0

D. Additional Embodiment: WX200

An ongoing experiment applies the pipeline to the five-DoF WX200 serial manipulator. This setting changes the robot mechanism while retaining the skill-learning and composition schedule. Figure 4 reserves the setup and rollout illustrations. Quantitative success rates will be added after the evaluation is complete.

E. Scope of the Evidence

The complete-task result comes from one SmolVLA training lineage and a development validation panel used during training. The retention results summarize a small number of rounds, and the LIBERO results are descriptive comparisons. Neither study isolates every stage of the pipeline. Earlier assisted training is part of the inherited CDPR policy; the reported full-task evaluation uses no assistance.

V. Conclusion

We presented a VLA adaptation pipeline that acquires skills with dense feedback, refines and consolidates them, and learns complete-task execution through milestone credit. With conditional latent-action likelihoods and continued residual training, the CDPR controller reaches 70.61% strict success in simulation development validation. Skill tests show partial recovery of forgotten behavior, while OpenVLA-OFT experiments support intermediate feedback on LIBERO-Spatial. These results motivate treating exploration, retention, and composition as connected parts of adaptation.

VI. Future Work

We will complete the WX200 evaluation to test the pipeline on another embodiment. Further experiments will compare training stages under matched budgets, repeat training across independent seeds, and evaluate unseen scenes. Automatic phase detection and progress estimation could also reduce the need for manually designed reward predicates.

References

[1] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, S. Alibert, M. Cord, T. Wolf, and R. Cadene, "SmolVLA: A vision-language-action model for affordable and efficient robotics," *arXiv preprint arXiv:2506.01844*, 2025. [Online]. Available: https://arxiv.org/abs/2506.01844

[2] M. J. Kim, C. Finn, and P. Liang, "Fine-tuning vision-language-action models: Optimizing speed and success," *arXiv preprint arXiv:2502.19645*, 2025. [Online]. Available: https://arxiv.org/abs/2502.19645

[3] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo, "DeepSeekMath: Pushing the limits of mathematical reasoning in open language models," *arXiv preprint arXiv:2402.03300*, 2024. [Online]. Available: https://arxiv.org/abs/2402.03300

[4] Q. Yu, Z. Zhang, R. Zhu *et al.*, "DAPO: An open-source LLM reinforcement learning system at scale," *arXiv preprint arXiv:2503.14476*, 2025. [Online]. Available: https://arxiv.org/abs/2503.14476

[5] H. Li, Y. Zuo, J. Yu, Y. Zhang, Z. Yang, K. Zhang, X. Zhu, Y. Zhang, T. Chen, G. Cui, D. Wang, D. Luo, Y. Fan, Y. Sun, J. Zeng, J. Pang, S. Zhang, Y. Wang, Y. Mu, B. Zhou, and N. Ding, "SimpleVLA-RL: Scaling VLA training via reinforcement learning," *arXiv preprint arXiv:2509.09674*, 2025. [Online]. Available: https://arxiv.org/abs/2509.09674

[6] Y. Guo, J. Zhang, X. Chen, X. Ji, Y.-J. Wang, Y. Hu, and J. Chen, "Improving vision-language-action model with online reinforcement learning," *arXiv preprint arXiv:2501.16664*, 2025. [Online]. Available: https://arxiv.org/abs/2501.16664

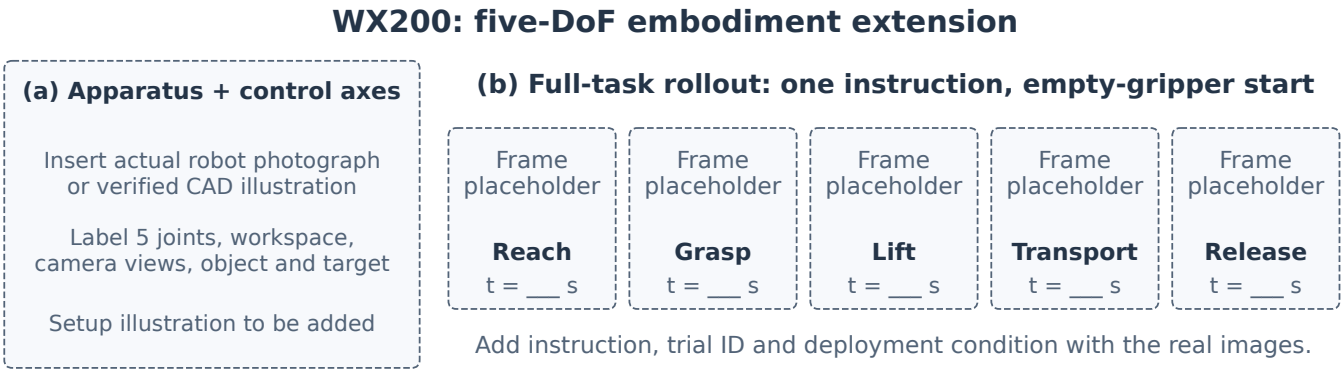


Fig. 4. WX200 evaluation layout. (a) Apparatus, workspace, cameras, and control axes. (b) Reach, grasp, lift, transport, and release under one instruction. The current panels reserve space for setup images and timestamped rollout frames from the ongoing experiment.

[7] W. Xiao, H. Lin, A. Peng, H. Xue, T. He, Y. Xie, F. Hu, J. Wu, Z. Luo, L. Fan, G. Shi, and Y. Zhu, "Self-improving vision-language-action models with data generation via residual RL," *arXiv preprint arXiv:2511.00091*, 2025. [Online]. Available: https://arxiv.org/abs/2511.00091

[8] Y. Fujita and S.-i. Maeda, "Clipped action policy gradient," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 80, 2018, pp. 1597–1606. [Online]. Available: https://proceedings.mlr.press/v80/fujita18a.html

[9] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, "LIBERO: Benchmarking knowledge transfer for lifelong robot learning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: https://arxiv.org/abs/2306.03310